**LAPORAN**
**WORKSHOP Mobile Web Server**
**(TIF330805)**
**Acara 20**



**Nama Tugas**
**API AUTHENTICATION DAN ERROR HANDLING**

**Nama:**
**Adam Ridho Saifulloh**

**NIM:**
**E41250216**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)**
**TEKNIK INFORMATIKA KAB. NGANJUK**
**JURUSAN TEKNOLOGI INFORMASI**
**POLITEKNIK NEGERI JEMBER**
**TAHUN 2026**

# A. PROSEDUR KERJA

Prosedur kerja Acara 20 melengkapi fondasi autentikasi dari Acara 19. Ditambahkan class Auth pada lapisan Core untuk membuat dan memvalidasi token JWT, aksi login pada AuthController, pemetaan resource login pada Router, proteksi pada MahasiswaController, serta penanganan error dengan try-catch pada lapisan Model.

### Langkah 1 — Membuat Class Auth pada Lapisan Core

Class Auth dibuat pada app/core/Auth.php dan berisi empat method: generateToken() untuk membuat token JWT (HS256) dengan masa berlaku, validateToken() untuk memeriksa tanda tangan dan masa berlaku token, requireAuth() sebagai middleware yang membaca header Authorization: Bearer lalu menolak request dengan kode 401 bila token tidak ada atau tidak valid, serta base64url() sebagai fungsi bantu. Seluruhnya memakai fungsi bawaan PHP (hash_hmac, base64_encode, hash_equals) tanpa pustaka tambahan.

**File:** app/core/Auth.php

```php
<?php
class Auth
{
    // Untuk praktik lokal. Pada aplikasi nyata, simpan secret di konfigurasi aman.
    const SECRET = 'kunci_rahasia_workshop_2026';

    private static function base64url(string $data): string
    {
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }

    private static function base64urlDecode(string $data)
    {
        $remainder = strlen($data) % 4;
        if ($remainder > 0) {
            $data .= str_repeat('=', 4 - $remainder);
        }

        return base64  decode(strtr($data, '-  ', '+/'), true);
    }

    public static function generateToken(array $payload): string
    {
        $header = self::base64url(json_encode([
            'alg' => 'HS256',
            'typ' => 'JWT',
        ]));

        $payload['exp'] = time() + 3600;
        $payloadEnc = self::base64url(json_encode($payload));
```

```
        $signature = self::base64url(hash_hmac(
            'sha256',
            "$header.$payloadEnc",
            self::SECRET,
            true
        ));

        return "$header.$payloadEnc.$signature";
    }

    public static function validateToken(string $token)
    {
        $parts = explode('.', $token);
        if (count($parts) !== 3) {
            return false;
        }

        [$header, $payload, $signature] = $parts;
        $expectedSignature = self::base64url(hash_hmac(
            'sha256',
            "$header.$payload",
            self::SECRET,
            true
        ));

        if (!hash_equals($expectedSignature, $signature)) {
            return false;
        }

        $decodedPayload = self::base64urlDecode($payload);
        if ($decodedPayload === false) {
            return false;
        }

        $data = json_decode($decodedPayload, true);
        if (!is_array($data) || !isset($data['exp']) || $data['exp'] < time()) {
            return false;
        }

        return $data;
    }

    public static function requireAuth(): array
    {
        $headers = function_exists('getallheaders') ? getallheaders() : [];
        $authorization = '';

        foreach ($headers as $name => $value) {
```

```
            if (strtolower($name) === 'authorization') {
                $authorization = $value;
                break;
            }
        }

        if ($authorization === '' && isset($_SERVER['HTTP_AUTHORIZATION'])) {
            $authorization = $_SERVER['HTTP_AUTHORIZATION'];
        }

        if (!preg_match('/^Bearer\s+(\S+)$/i', trim($authorization), $matches)) {
            Response::json(401, false, 'Token tidak ditemukan');
        }

        $payload = self::validateToken($matches[1]);
        if (!$payload) {
            Response::json(401, false, 'Token tidak valid atau kedaluwarsa');
        }

        return $payload;
    }
}
```

**Langkah 2 — Menambahkan Aksi login() pada AuthController**

Aksi login() memeriksa kelengkapan username dan password (400), mencari user berdasarkan username, lalu mencocokkan password dengan password_verify(). Bila user tidak ditemukan atau password salah, dikembalikan kode 401 dengan pesan yang sama agar tidak membocorkan informasi akun. Bila cocok, dibuat token berisi id dan role user.

**File:** app/controllers/AuthController.php (bagian yang ditambahkan)

```
public function login(): void
    {
        $data = json_decode(file_get_contents('php://input'), true);

        if (!is_array($data) || empty($data['username']) || empty($data['password'])) {
            Response::json(400, false, 'Username dan password wajib diisi');
        }

        $user = $this->model->findByUsername(trim($data['username']));

        if (!$user || !password_verify($data['password'], $user['password'])) {
            Response::json(401, false, 'Username atau password salah');
        }

        $token = Auth::generateToken([
            'id' => (int) $user['id'],
```

```
            'role' => $user['role'],
        ]);

        Response::json(200, true, 'Login berhasil', [
            'token' => $token,
            'role' => $user['role'],
        ]);
    }
}
```

### Langkah 3 — Mendaftarkan Resource login pada Router

Resource login ditambahkan pada peta resource Router. Endpoint register dan login diproses bersama pada satu blok khusus karena keduanya bukan CRUD standar dan hanya menerima method POST (method lain dijawab 405).

**File:** app/core/Router.php (bagian yang diubah)

```
private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',   // Tugas Praktik Mandiri 1
        'register'  => 'AuthController',
        'login'     => 'AuthController',
    ];
```

```
// Endpoint autentikasi bukan bagian dari CRUD standar.
        if (in_array($resource, ['register', 'login'], true)) {
            if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
                header('Allow: POST');
                Response::json(405, false, 'Method tidak diizinkan');
            }

            if ($resource === 'register') {
                $controller->register();
            } else {
                $controller->login();
            }
            return;
        }
```

### Langkah 4 — Melindungi Method Controller dengan Auth::requireAuth()

Auth::requireAuth() dipanggil pada baris pertama method store(), update(), dan destroy() di MahasiswaController, sedangkan index() (GET) tetap publik. Contoh pada method store():

**File:** app/controllers/MahasiswaController.php (method store)

```
// POST /mahasiswa
    public function store(): void
```

```
    {
        Auth::requireAuth();
        $data = $this->validate($this->input());

        try {
            $id = (int) $this->model->create($data);
        } catch (DomainException $e) {
            $this->fail(409, $e->getMessage());
        }

        $this->json(201, [
            'success' => true,
            'message' => 'Data berhasil ditambahkan',
            'data' => ['id' => $id] + $data
        ]);
    }
```

**Langkah 5 — Menerapkan Try-Catch pada Lapisan Model**

Pada Model Mahasiswa, proses insert dibungkus try-catch. Apabila terjadi PDOException, server tidak membocorkan detail kesalahan database, melainkan mengembalikan kode 500 dengan pesan umum melalui Response::json(). Pengecekan NIM duplikat dilakukan sebelum insert dan dilaporkan sebagai DomainException yang diterjemahkan Controller menjadi kode 409.

**File:** app/models/Mahasiswa.php (method create)

```
public function create(array $data): string {
        if ($this->nimExists($data['nim'])) {
            throw new DomainException('NIM ' . $data['nim'] . ' sudah terdaftar');
        }
        try {
            $stmt = $this->db->prepare(
                'INSERT INTO mahasiswa (nim, nama, prodi) VALUES (:nim, :nama, :prodi)'
            );
            $stmt->execute([
                ':nim'   => $data['nim'],
                ':nama'  => $data['nama'],
                ':prodi' => $data['prodi'],
            ]);
            return $this->db->lastInsertId();
        } catch (PDOException $e) {
            Response::json(500, false, 'Terjadi kesalahan pada server');
        }
    }
```

**Langkah 6 — Pengujian Alur Autentikasi**

Pengujian dilakukan pada Postman dengan Environment Local. Variabel yang dipakai ditunjukkan pada Tabel 1. Request POST {{base_url}}/login memiliki script test yang otomatis menyimpan token hasil login ke variabel environment token, sehingga request lain cukup memakai header Authorization: Bearer {{token}}.

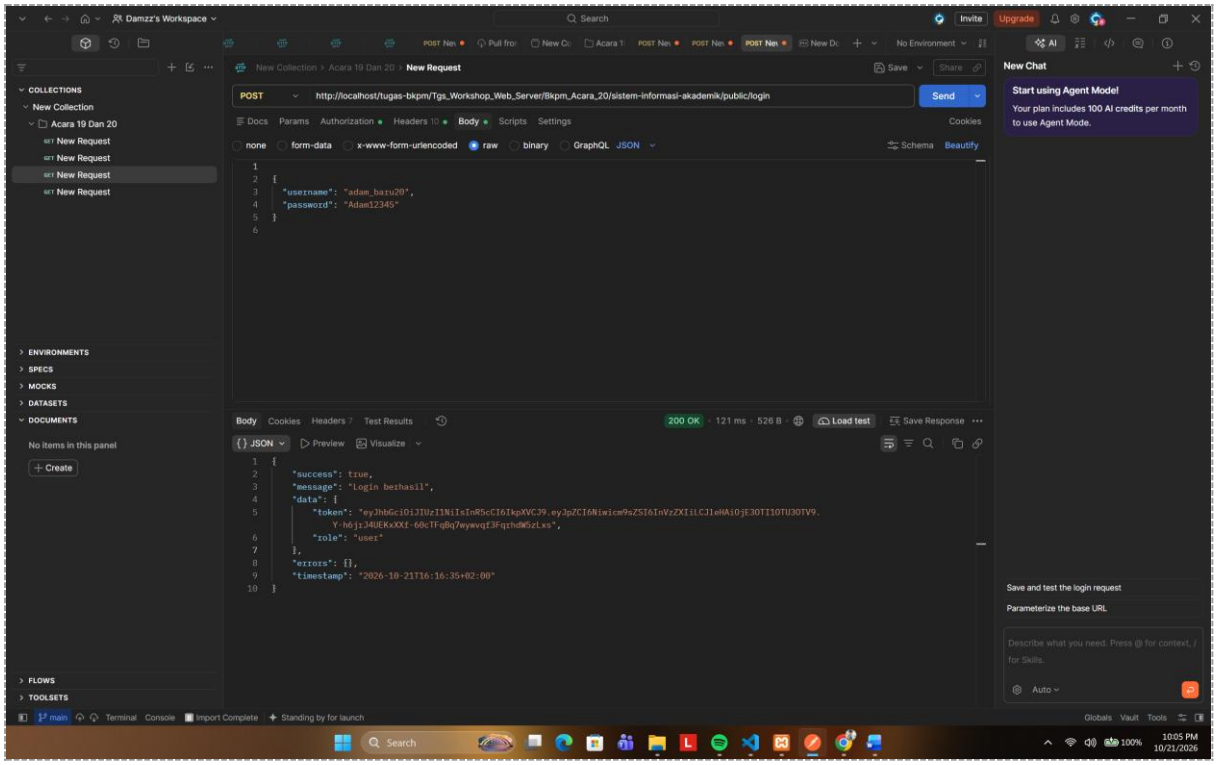
| Variabel | Nilai | Fungsi |
|---|---|---|
| base_url | http://localhost/sistem-informasi-akademik/public | Alamat dasar API |
| auth_username | adam_test | Akun hasil registrasi Acara 19 |
| auth_password | Rahasia123! | Password akun tersebut |
| token | (terisi otomatis setelah login) | Menyimpan token JWT |
| invalid_token | token_palsu | Token sengaja salah untuk uji gagal |

*Tabel 1. Variabel Environment Local*

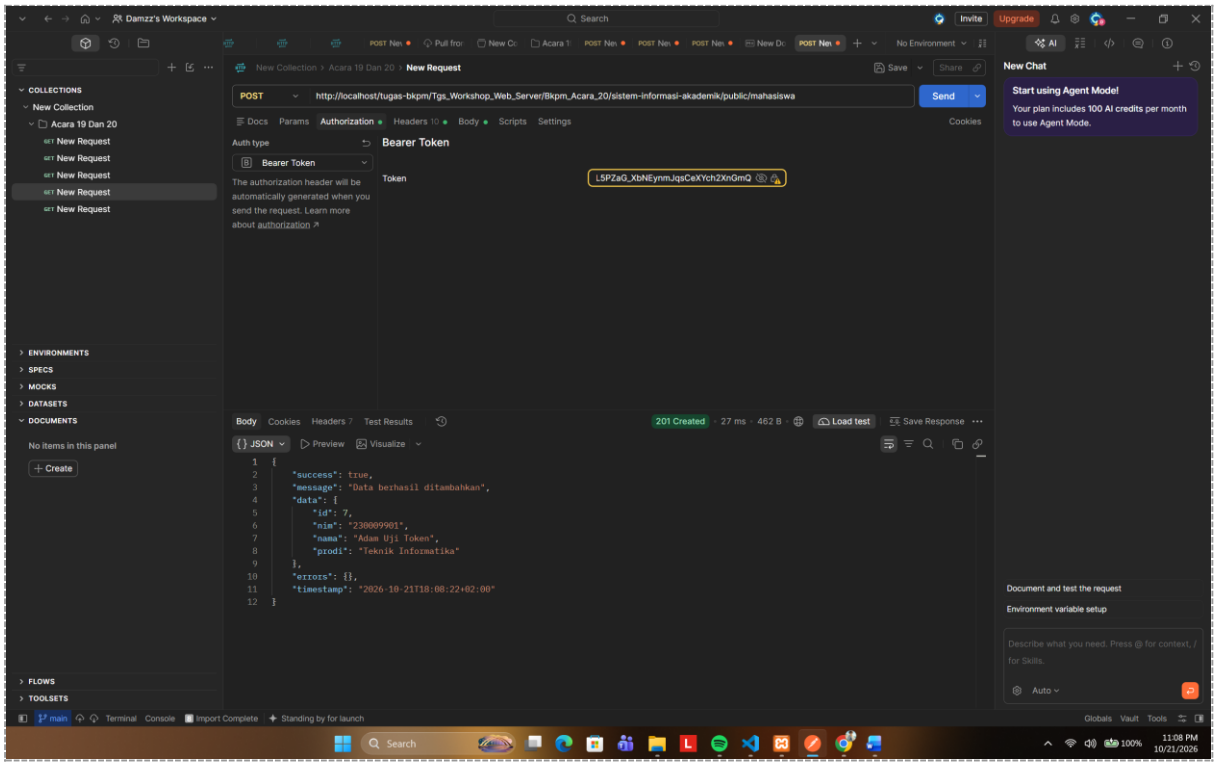
Skenario pengujian ditunjukkan pada Tabel 2.

| No | Skenario | Request | Status | Keterangan |
|---|---|---|---|---|
| 1 | Login dengan akun hasil registrasi | POST /login, body username dan password | 200 | Token diterima |
| 2 | Tambah mahasiswa dengan token valid | POST /mahasiswa, Bearer {{token}} | 201 | Data tersimpan |
| 3 | Tambah mahasiswa tanpa token | POST /mahasiswa, tanpa header Authorization | 401 | Token tidak ditemukan |
| 4 | Tambah mahasiswa dengan token diubah | POST /mahasiswa, Bearer token_palsu | 401 | Token tidak valid |

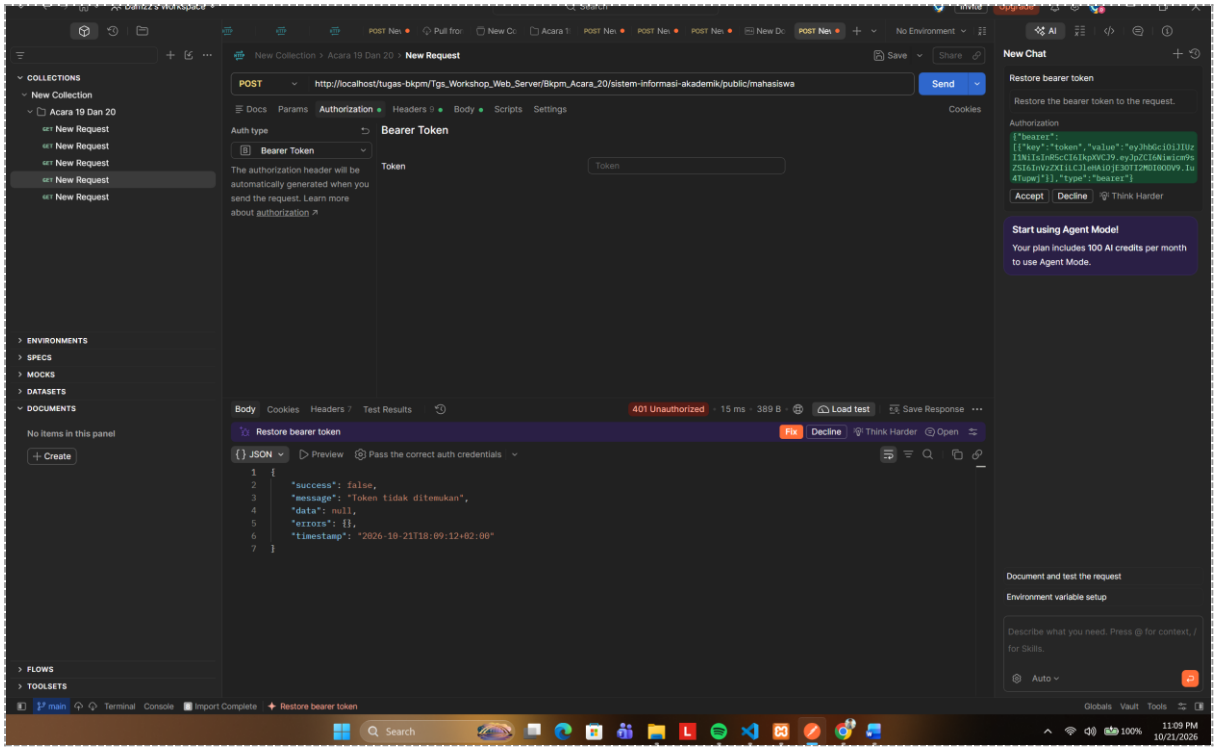
*Tabel 2. Skenario pengujian alur autentikasi*



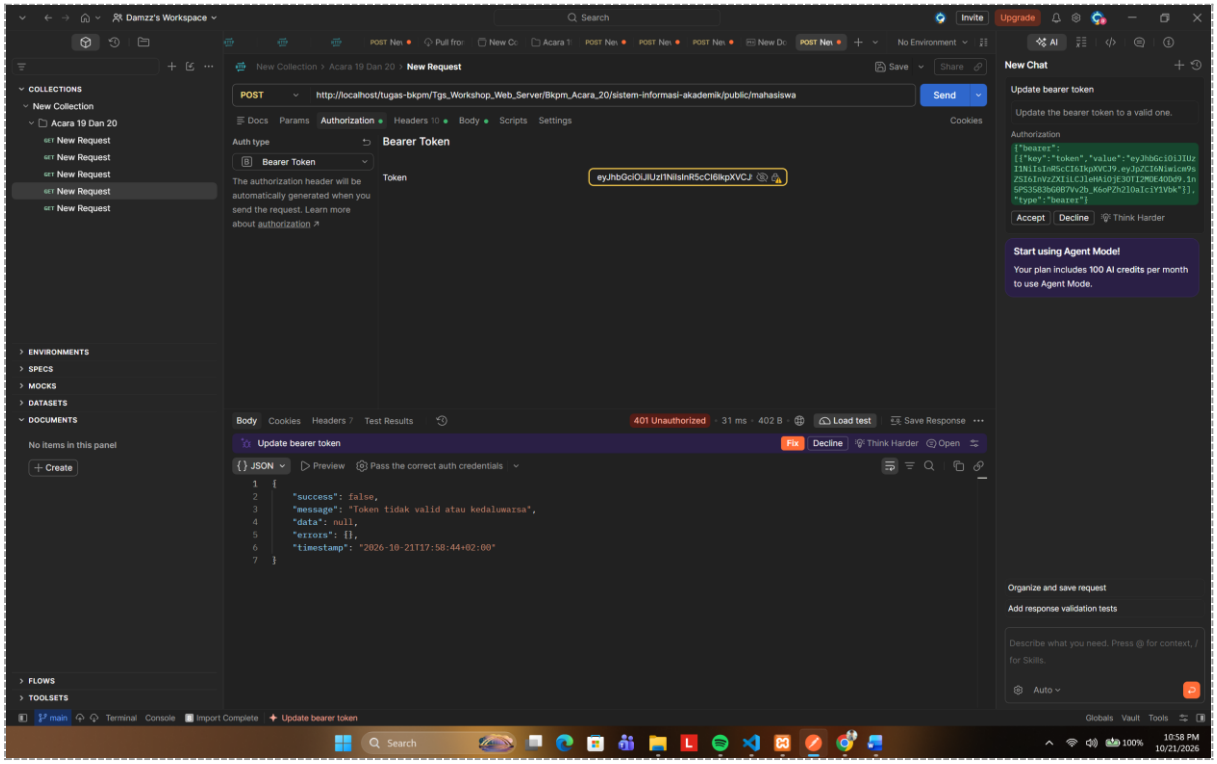

*Gambar 1. Pengujian login berhasil*




*Gambar 2. Pengujian endpoint terproteksi dengan token valid*




*Gambar 3. Pengujian tanpa token*




*Gambar 4. Pengujian dengan token yang diubah*

# B. TUGAS PRAKTIK MANDIRI

### Tugas 1: Diskusi JWT Stateless dan Proses Logout

Pada JWT, server tidak menyimpan sesi maupun daftar token yang telah diterbitkan. Keabsahan token hanya diperiksa dari dua hal: kecocokan signature (dihitung ulang dengan

secret milik server) dan klaim exp (masa berlaku). Karena tidak ada data sesi di server, tidak ada yang dapat "dihapus" ketika pengguna logout. Selama signature benar dan exp belum lewat, token tetap diterima oleh Auth::validateToken(), termasuk jika token tersebut telah disalin atau dicuri pihak lain.

Dengan demikian logout pada JWT murni hanya terjadi di sisi client, yaitu dengan membuang token yang tersimpan. Agar token dapat benar-benar dicabut dari sisi server, dibutuhkan mekanisme tambahan, antara lain:

- Blacklist token: server menyimpan identitas token yang sudah logout (misalnya pada tabel atau cache) sampai masa berlakunya habis, dan Auth::requireAuth() menolak token yang ada di daftar tersebut.
- Masa berlaku token pendek: token berumur beberapa menit sehingga dampak token yang bocor terbatas, biasanya dipadukan dengan refresh token yang dapat dicabut.
- Mengganti secret: seluruh token yang beredar menjadi tidak valid, namun semua pengguna ikut terdampak sehingga hanya cocok untuk keadaan darurat.

**Kesimpulan:** sifat stateless membuat JWT ringan dan mudah diskalakan karena server tidak perlu menyimpan sesi, tetapi harga yang dibayar adalah ketidakmampuan mencabut token sebelum kedaluwarsa. Penambahan blacklist mengembalikan sebagian sifat stateful ke server, sehingga pemilihan antara masa berlaku pendek dan blacklist bergantung pada tingkat keamanan yang dibutuhkan aplikasi.

### Tugas 2: Memperpendek Masa Berlaku Token Menjadi 5 Menit

Masa berlaku token pada Auth::generateToken() diubah dari 3600 detik (1 jam) menjadi 300 detik (5 menit) dengan mengganti nilai penambah pada klaim exp.

**File:** app/core/Auth.php (perubahan pada generateToken)

```
public static function generateToken(array $payload): string
{
    // ...
    $payload['exp'] = time() + 300; // masa berlaku 5 menit (sebelumnya 3600 = 1 jam)
    // ...
}
```

Pengujian dilakukan dengan login untuk mendapatkan token baru, mengirim POST /mahasiswa dengan token tersebut sebelum 5 menit (hasil yang diharapkan 201), lalu mengirim request yang sama setelah lebih dari 5 menit tanpa login ulang. Pada kondisi kedua, validateToken() menolak token karena exp sudah lewat sehingga server mengembalikan kode 401 dengan pesan "Token tidak valid atau kedaluwarsa". Untuk mendapatkan akses kembali, client harus login ulang dan memperoleh token baru.